INFERENCE AND KNOWLEDGE PROCESSING

Introduction

In Artificial Intelligence (AI) and Expert Systems, reasoning is the foundation of intelligent behavior. It is what allows machines to draw new conclusions or make informed decisions from existing knowledge. While data and facts describe what is known, *reasoning* helps the system infer what is not yet known; just as humans do when they analyze information and make judgments.

For example, if a medical expert system knows that *"If a patient has malaria, they will test positive for plasmodium"* and it receives a fact that *"John tests positive for plasmodium,"* the system can reason and conclude that *"John may have malaria."*

This act of deriving new knowledge from known facts is what we call **inference**.

The Process of Reasoning

Reasoning is the process of drawing conclusions or inferences; moving from what is known (facts) to what is unknown (inferred knowledge). It is the heart of intelligent systems because it connects stored knowledge to new situations.

When a knowledge-based system reasons, it uses the information in its knowledge base (facts, rules, and relationships) to answer questions, solve problems, or make predictions.

There are several major types of reasoning, each with its own logic and applications:

- Deductive reasoning
- Inductive reasoning
- Abductive reasoning

Deductive Reasoning

Deductive reasoning starts from general statements or premises and derives specific conclusions that logically follow.

It guarantees logical correctness: if the premises are true and reasoning is valid, the conclusion must be true.

Example:

If *all humans are mortal*, and *Socrates is a human*, then *Socrates is mortal.*

The reasoning is logically valid because the conclusion follows necessarily from the premises.

However, deductive reasoning can sometimes be valid but not true, depending on the premises used.

For instance:

If it is raining, then the ground is dry.
It is raining.
Therefore, the ground is dry.

The reasoning is valid (it follows the structure of deduction), but it is *not true in reality* because the premise contradicts our knowledge of the world.

Thus, deductive reasoning depends on the truth of its premises and the correctness of its logical structure.

Inductive Reasoning

Inductive reasoning moves from specific observations to general conclusions.
It is the process of generalizing patterns seen in a few cases to predict or describe future ones.

Example:

Every elephant you have seen has a trunk.
Therefore, you conclude that *all elephants have trunks.*
Induction is common in learning and science because it forms the basis for hypothesis formation.
However, inductive reasoning is not logically certain — it only suggests what is *probably* true based on available evidence.

Abductive Reasoning

Abductive reasoning goes in the reverse direction: it moves from an observed fact to the most likely explanation or cause.
It is reasoning *to the best explanation.*

Example:

Sam is driving too fast.
Whenever Sam drinks, she drives too fast.
Therefore, she has probably been drinking.
This reasoning gives a plausible cause but is not always reliable — Sam could be speeding for other reasons.
Abduction is used widely in diagnostic systems, such as medical or mechanical troubleshooting, where the system tries to infer the most likely cause of an observed symptom.

Types of Reasoning

Reasoning can take several practical forms in communication and problem solving. These include reasoning from signs, cause and effect, analogy, and examples.

1. **Reasoning from Signs**

Here, we infer information from a **sign, symbol, or indicator** that represents something specific.

Example:

If you make a call and hear a busy signal, you infer that someone is already on the line.
However, one must be cautious to avoid stereotypes. For instance, assuming a person behaves a certain way just because of their tribe or background is a misuse of reasoning from signs.

2. **Cause and Effect Reasoning**

This type of reasoning assumes that one event causes another.

Example:

“I was late because my alarm did not go off.”
Or,
“If I don’t study, I will fail the test.”

Care must be taken not to assume that sequential events are always causally related.
For example, "I got sick after eating mangoes" does not mean the mangoes caused the illness.

3. **Reasoning by Analogy**

This draws parallels between similar events, people, or objects.
Example:
All mothers are women; Ann is a mother; therefore, Ann is a woman.
However, reasoning by analogy does not always work backward. From "Ann is a woman," we cannot conclude that she is a mother.

4. **Reasoning by Example**

This involves basing conclusions on facts, data, or illustrations drawn from examples.
Example:
A survey shows that Candidate Mende has 40% approval, higher than his rival. Therefore, he is likely to win the election.
The key question in reasoning by example is: *"Do I have enough facts to support this conclusion?"*

Reasoning Methods

Reasoning is closely tied to knowledge representation, which allows facts and relationships to be stored in a structured way. Once knowledge is represented, inference applies logical rules to derive new conclusions.
In AI, reasoning methods are divided into two broad categories:

1. Deductive Reasoning
2. Inductive Reasoning

1. **Deductive Reasoning**

Deductive reasoning is the most formal and is used in propositional and predicate logic.
It starts from a set of axiom**s** (commonly accepted truths) and applies **inference rules** to derive conclusions.
Let's illustrate with an example:

Akidiva is a man.
Akidiva is a Luhya.
All Luhyas are Kenyans.
All Kenyans are either loyal to Kibaki or hate him.
Everyone is loyal to someone.
People only cheat leaders they are not loyal to.
Akidiva tried to cheat Kibaki.

From these premises, the inference engine can deduce whether Akidiva is loyal or disloyal to Kibaki using deductive logic.

1) man (akidiva)
2) luhya (akidiva)
3) ∀ X (luhya (X) → kenyan (X))
4) leader (kibaki)
5) ∀ X (kenyan(X) → loyal_to (X, kibaki) V hate (X, kibaki)
6) ∀ X ∃ Y((person (X) → person (Y) ∧ loyal_to (X,Y))
7) ∀ X ∀ Y (person (X) ∧ person (Y) ∧ try_to_cheat (X,Y) → ¬loyal_to (X,Y)
8) try_to_cheat (akidiva, kibaki)
9) ∀ X (man (X) → person (X))

Process of Deductive Reasoning

There are two main parts:

1. **Single Inference:**
   Applying one inference rule to two or more facts to derive a new conclusion.

2. **Multiple Inference:**
   Repeatedly applying inference rules across the entire knowledge base until a final conclusion is reached.

Together, these form the knowledge processing capability of an intelligent system.

Inference in Propositional Logic

Propositional logic deals with simple statements that are either true or false.
There are several rules of inference that allow us to combine known truths to produce new ones, such as:

- **Modus Ponens:** If (P → Q) and P are true, then Q is true.
- **Modus Tollens:** If (P → Q) and ¬Q are true, then ¬P is true.
- **Hypothetical Syllogism:** If (P → Q) and (Q → R), then (P → R).

A rule of inference is valid if it produces a tautology; a statement that is always true regardless of the truth values of its parts.

Inference in Predicate Calculus

Predicate calculus extends propositional logic by using predicates and quantifiers such as:

- **Universal quantifier (∀)** – meaning "for all"
- **Existential quantifier (∃)** – meaning "there exists"

This allows for more expressive reasoning.
For example:

$\forall x$ (Human(x) → Mortal(x))
Human(Socrates)
∴ Mortal(Socrates)
Here, quantifiers allow reasoning about all entities in a domain, not just individual statements.

Rules of Inference

Rules of Inference

Law of Detachment: p -> q
p
therefore, q

Law of the Contrapositive: p -> q
therefore, ~q -> ~p

Chain rule (Low of the Syllogism): p -> q
q -> r
therefore, p -> r

Law of Disjunctive Inference: p v q p v q
~p ~q
therefore, q p

Law of Double Negation: ~(~p)
therefore, p

DeMorgan's Law: ~(p ^ q) ~(p v q)
therefore, ~p v ~q ~p ^ ~q

Law of Simplification: p ^ q ~(p v q)
therefore, p q

Law of Conjunction: p
q
therefore, p ^ q

Law of Disjunctive Addition: p
therefore, p v q

Law of Conjunctive Argument: ~(p ^ q) ~(p ^ q)
p q
therefore, ~q ~p

Resolution in First-Order Logic

Resolution is a method of proof by contradiction.

To prove a statement X is true, the system assumes ¬X (not X) and shows that this assumption leads to a contradiction.

Example:

1. ¬Feathers(X) ∨ Bird(X) - "If X has feathers, then X is a bird."
2. Feathers(X) - "X has feathers."

By resolving (1) and (2), ¬Feathers(X) cancels out with Feathers(X), leaving Bird(X).

If the negation of Bird(X) also existed, it would create a contradiction - proving Bird(X) must be true.

Unification

Unification is the process of making two logical expressions identical by finding appropriate substitutions for their variables.

It is essential for applying resolution in predicate logic.

Takes two sentences in FOL and finds a substitution that makes them look the same required to prove theorems using resolution.

If two predicates are identical, then they match by definition.

If one or both contain variables, then appropriate substitutions must be found using unification as follows:-

- A variable can be replaced by a constant.
- A variable can be replaced by another variable.
- A variable can be replaced by a predicate, as long as the predicate does not contain that variable.
- Unification Example

Example:

1. hates(X, Y)
2. hates(Onyango, Kales)

Unifying (1) and (2) means substituting X = Onyango and Y = Kales.

Prolog and similar AI languages use this mechanism to match patterns and derive results.

Consider hates (X, Y)

- hates (Onyango, Kales)
- hates (Okeyo, Cabbage)

Unify 2 with 1 by binding Onyango to X and Kales Y, also unify 3 with 1 by binding Okeyo to X and Cabbage to Y.

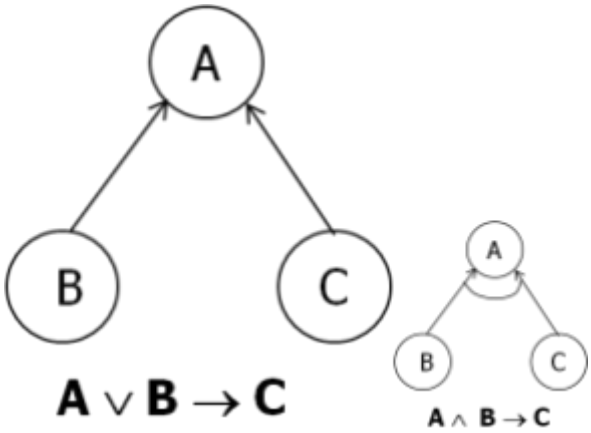
- Hates (X, Vegetable (Y))
- Hates (Onyango, Vegetable (Kales))
- Hates (Z, Kales)

Unify 6 with 1- X with Z, Y with Kales.
Unify 4 and 5, bind Onyango to X, Kales to Y.
Note: Prolog uses a generalized version of unification algorithm called match.

Multiple Inference in Deductive Reasoning
Multiple inference involves **deciding which rule or predicate to apply next** when several are possible.
This can be represented using **graphs, trees, and and/or structures.**
Graphs

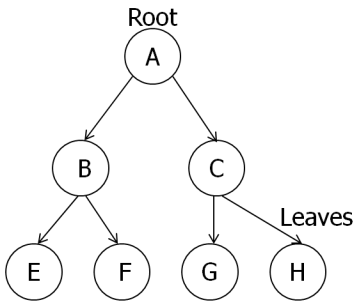
- Represent knowledge using nodes (concepts) and arcs (relationships).
- A **directed graph** has arrows showing direction of reasoning.
- A **path** connects nodes; a **cycle** occurs when a path leads back to its starting node.

Rules and predicates are shown with nodes and arcs.
The two arcs in the AND diagram are connected by another arc signifying that both A and B must be true to reach conclusion C from this rule.



Trees

- A directed graph with no cycles which starts with a beginning node (root) and ends with ending nodes (leaves).
- Paths start from the root and end with leaves in a tree There is only one path from the root to each leaf.
- Used extensively in A.I. And Expert Systems e.g. and/or graph and decision tree.



Directed graph, with branches,

- A - B - E etc.

Backward and Forward Chaining

These are the two main reasoning control strategies used in rule-based expert systems.

Forward Chaining (Data-Driven)

Forward chaining begins with known facts and applies inference rules to discover all possible conclusions.

Each time a rule's conditions (the *IF* part) match the facts, the rule fires, and its conclusion (the *THEN* part) is added to the knowledge base.

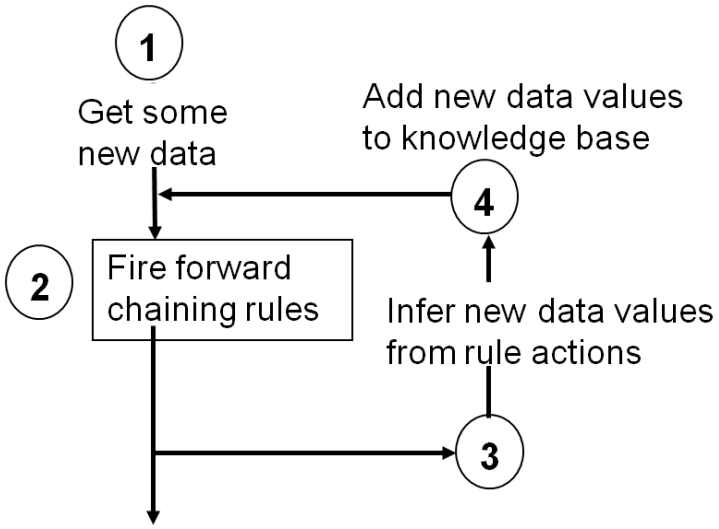
The system requires the user to provide facts pertaining to the problem.

The inference engine tries to match each fact with the antecedent or if part of a rule. If the match succeeds, the rule fires and the consequent of that rule is established and is added to the known facts of the case. The process continues until the inference engine has drawn all possible conclusions by matching facts to antecedents of rules in the knowledge base.

Among these could be goals of the system.

The Forward Chaining algorithm

1. Enter new data
2. Fire forward chaining rules
3. Rule actions infer new data values
4. Go to step 2
5. Repeat until no new data can be inferred
6. If no solution, rule base is insufficient



Algorithm (Simplified):

1. Enter new data (facts).
2. Check rules whose conditions match the facts.
3. Apply those rules to generate new facts.
4. Repeat until no new facts can be inferred.
5. Reasons for forward chaining

When everything that can possibly be concluded about a set of data needs to be known.
Many conclusions are possible from a single data item.
It is important to communicate new conclusions to the user immediately.

Characteristics of forward chaining:

1. Good for monitoring, planning, and control
2. Looks from present to future.
3. Works from antecedent to consequent.
4. Is data-driven, bottom-up reasoning.
5. Works forward to find what solutions follow from the facts.
6. It facilitates a breadth-first search.
7. The antecedents determine the search.
8. It does not facilitate explanation.

Used for: monitoring, control, and planning systems.
Example: Fire detection system — starts with sensor data and infers danger levels.

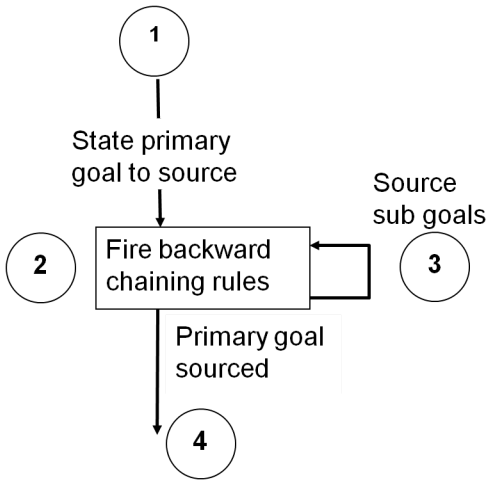
Backward Chaining (Goal-Driven)

Backward chaining starts with a **goal** and works backward to find facts that support it.

The inference engine searches for rules that could produce the goal as a conclusion, then tests their conditions recursively.

The Backward Chaining algorithm

1. State a specific goal (question)
2. Find rules which resolve the goal
3. At runtime, answer questions to satisfy the antecedents of the rules as required
4. Obtain a result (goal resolved or not)



The goal is to establish whether G is true.

The inference engine checks to see which rule has G as its consequent.

- Rule 4 fires

To conclude from this that G is true, E or F must be shown to be true.

E becomes the current goal of the system

The inference engine checks to see which rule has E as its consequent

- Rule 2 fires

The current goal becomes D

The inference engine checks to see which rule has D as its consequent There is none.

It asks the user to provide information about D

D is True

The inference engine concludes that E is true and G is also true.

D is False The system fails to establish the truth of E

In this case, it backtracks and picks up F to get at the goal G.

B and C become the current goal. B is not the consequent of any rule and the system asks the user if it is true.

B is False

- No matter what C is, the conditional part of rule 3 is false and the inference engine fails to establish whether G is true.

B is True

- The inference engine pursues C etc

Reasons for Backward Chaining

- There is a clear set of statements which must be confirmed or denied.
- A large number of questions could be asked of the user, but typically only a few are necessary to resolve a situation.
- It is desirable to have interactive dialogue with the user.
- Rule execution depends on data gathering which may be expensive or difficult.

Characteristics of backward chaining:

- Good for Diagnosis.
- Looks from present to past.
- Works from consequent to antecedent.
- Is goal-driven, top-down reasoning.
- Works backward to find facts that support the hypothesis.
- It facilitates a depth-first search. The consequents determine the search.
- Facilitates explanation.

Used for: diagnostic or advisory systems, such as medical diagnosis or fault finding.

Comparison: Backward vs. Forward chaining

Backward Chaining

- Useful when the number of goals is small and the and/or graph does not have numerous levels.
- In some systems, the user has the choice of telling the inference engine which goal it must pursue - Goal selection by the user.

Forward Chaining Performs well when the number of goals is large, the user has a given set of facts at the start and wants to find the implications of these facts.

Feature	Forward Chaining	Backward Chaining
Direction	From facts to conclusions	From goals to supporting facts
Type	Data-driven	Goal-driven
Search Type	Breadth-first	Depth-first
Suitable For	Monitoring, planning	Diagnosis, problem-solving

Inductive Reasoning

While deductive reasoning is the foundation of logic, most *learning* in AI is inductive — discovering general rules from specific examples.

Inductive reasoning allows systems to **learn from experience** and improve over time.

Example:
After observing several successful loan applications, an AI system may infer that "applicants with stable income and no past defaults are likely to repay loans."
This generalization becomes part of the system's decision rules.

Methods for Inductive Reasoning
Inductive reasoning is the process of drawing **general conclusions from specific examples or observations**. Unlike deductive reasoning, which guarantees truth, inductive reasoning deals with **probable truth**—it predicts or generalizes based on patterns found in data.
In Artificial Intelligence (AI), inductive reasoning forms the foundation for **learning systems** — systems that improve performance by learning from experience.

1. Decision Trees

Decision trees are one of the most popular and intuitive methods for inductive reasoning.
They are used when we have **data with known outcomes** and want the system to learn the pattern that links input features to results.

- Each **internal node** represents a test or condition on an attribute (for example, "Is the applicant's income greater than $50,000?").
- Each **branch** represents the outcome of that test (Yes/No).
- Each **leaf node** represents a final decision or class label (for example, "Approve Loan" or "Reject Loan").

Decision trees are used in **classification** and **prediction** tasks such as:

- Determining whether to approve a bank loan,
- Predicting whether an email is spam,
- Diagnosing a disease based on symptoms.

They are easy to interpret because they mimic human reasoning through a sequence of questions and decisions.
Must have a set of representative cases for which the goal or decision value is known E.g. the following data for the possible cases in the Mortgage Loan Case.

Steady job	Adequate Assets	Adequate income	Approve Loan
Yes	Yes	Yes	Yes
Yes	No	Yes	Yes
No	Yes	Yes	Yes
No	No	Yes	No
Yes	Yes	No	No
Yes	No	No	No
No	Yes	No	No
No	No	No	No

Advantages:

- Simple to understand and visualize.
- Requires little data preparation.

- Can handle both categorical and numerical data.

Limitations:

- Small changes in data can lead to a completely different tree.
- Prone to **overfitting** (becoming too specific to the training examples).

2. ID3 Algorithm

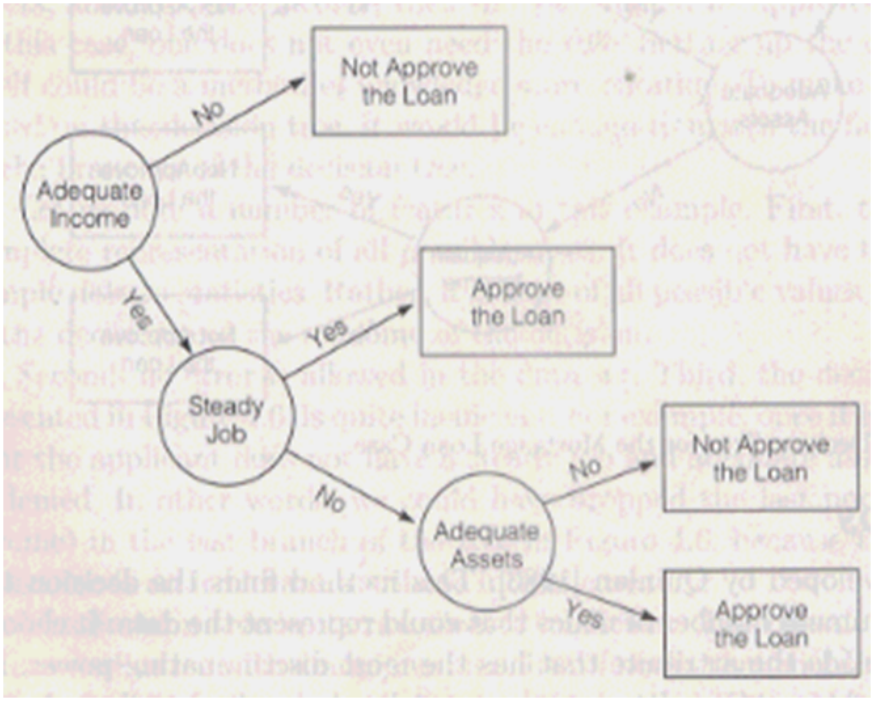
The tree in its raw form is inefficient. Try to find a tree with the minimum number of nodes. The **Iterative Dichotomiser 3 (ID3)** algorithm was developed by **Ross Quinlan** in the 1980s as a systematic method for building decision trees.

The main idea is to find the **simplest and most efficient tree** by selecting attributes that best divide the data into distinct classes.

It uses a concept called **Information Gain**, which measures how well an attribute separates the examples according to their target classification.

Steps of the ID3 Algorithm:

1. Select the attribute with the highest **information gain** (most informative attribute).
2. Split the data into subsets based on the attribute's possible values.
3. Repeat the process recursively for each subset until:
    - All instances in a subset belong to the same class, or
    - No more attributes remain.



Example:

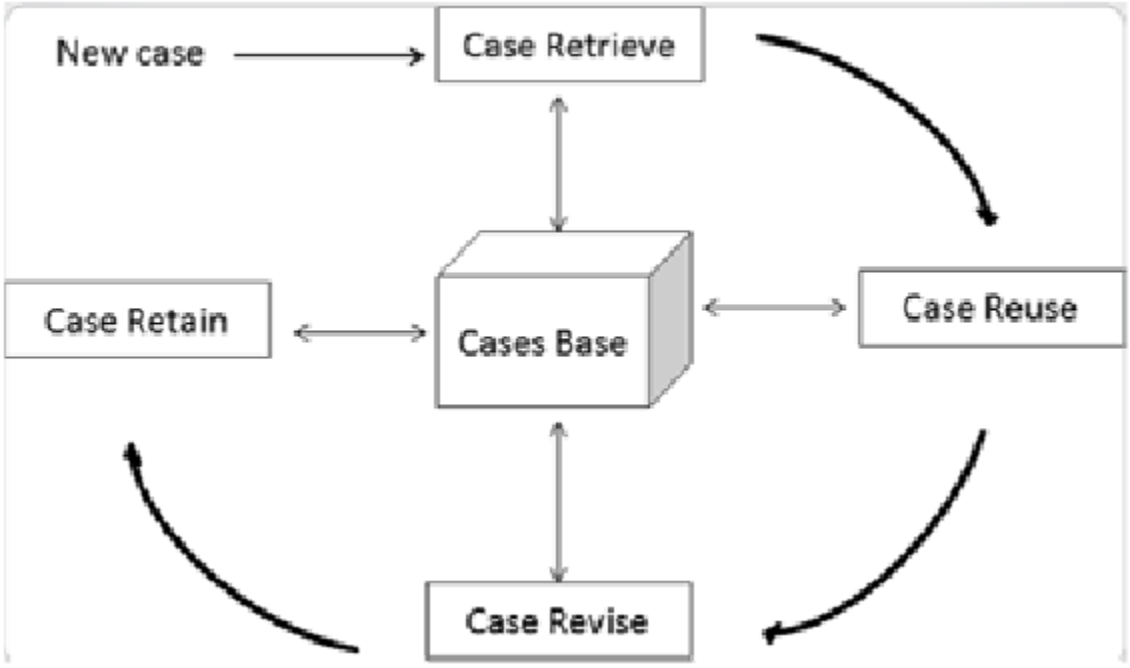
In a loan approval system, ID3 might find that "income level" is the best attribute to split on

first, followed by “credit score,” and so on.
The result is a compact decision tree that predicts loan approval accurately and efficiently.

3. Case-Based Reasoning (CBR)

Case-Based Reasoning is another major method of inductive reasoning.
It mimics human problem-solving behavior — we often solve new problems by recalling and adapting solutions from past similar experiences.
For example, a doctor may diagnose a new patient by remembering a past patient with similar symptoms.
Similarly, an AI system can store old “cases” (problems and their solutions) and use them to handle new ones.

The CBR Cycle



Case-Based Reasoning operates in a continuous four-stage cycle:

1. **Retrieve:**
   The system searches its knowledge base for past cases that are most similar to the current problem. The retrieve step provides information through its result, namely, how often a case is retrieved. Also, to improve performance the mean retrieval time may be stored. Maximum and minimum may also be of interest. Other usage information are the last update or last retrieval time. The queries may be logged here, too. In the maintenance phase the queries could be used to enhance the vocabulary, the similarity measures or the adaptation knowledge. They even may be used to seed new cases. Comparisons of retrieval results of different versions of the case–based reasoner regarding the stored queries may yield interesting insights into the behaviour of the system over time.

2. **Reuse:**
   The solution from the retrieved case is proposed for the new problem. The reuse step adapts the retrieved cases to solve the given problem. An adaptation counter can count which of the retrieved cases could be adapted to the query and is therefore suggested as a possible solution.

3. **Revise:**
   If necessary, the solution is tested and adjusted to better fit the new situation. If the suggested solution is rejected this may be counted as well as how often the case has been accepted as a solution. This information can help to decrease the case base size because it may be more favorable to remove an often rejected case from the case base than accepted cases.

4. **Retain:**
   After solving the problem, the new case (problem and solution) is stored in the case library for future use. This step adds adapted cases to the system. It would be advisable to mark those cases as unconfirmed. The user then has a choice between confirmed and unconfirmed cases. A second purpose of the retain step is to modify the similarity measures by modifying the indexing structures. Modifications like that should only be used by the case–based reasoner if there are possibilities to track the changes, or better measure the impact of those changes.

This process allows AI systems to learn incrementally; improving their performance over time as more cases are accumulated.

Applications:

- Medical diagnosis (comparing patient symptoms).
- Customer service (matching with previous complaint resolutions).
- Engineering design and maintenance.

Advantages:

- No need for complete theoretical models.
- Easy to update with new experiences.
- Encourages continual learning.

Limitations:

- Requires a well-organized case library.
- Performance depends on the quality of stored cases and similarity measures.

Criteria for choosing inferences

- The logical reasoning process.
- Design features of the system.

- The inputs and where they come from.
- The outputs and where they go.
- How the inputs and outputs map to forward or backward chaining.

Additional Types of Reasoning in AI

- **Heuristic reasoning:** Based on experience or rules of thumb.
- **Abductive reasoning:** Reasoning from effects to possible causes.
- **Default reasoning:** Assuming typical knowledge when information is missing.
- **Non-monotonic reasoning:** Allows previous conclusions to change when new facts appear.
- **Meta-level reasoning:** Reasoning about the reasoning process itself.